August 10, 2026

The results below are generated from an R script.

```r
library(ks)
### function for simulating Data
Sim.data <- function(N,mu_n,frac,delta=1,Seed=NULL){
  # N: number of clusters
  # mu_n: mean number of members per cluster
  # frac: fraction of the mean to determine the standard deviation of number of member per cluster (Var
  # delta: multiplier for the variance of random effects
  n <- round(rnorm(N,mu_n,mu_n*frac))
  n[n<10] = 10
  if(!is.null(Seed)){set.seed(Seed)}

  Beta <- matrix(c(450,300,500,500),1,4)
  D <- matrix(rep(0, times=16), ncol=4)
  diag(D) <- c(1, 1, 1, 1);D[1, 3] <- D[3, 1] <- .4; D[2, 4] <- D[4, 2] <- sqrt(.5)
  Sigma <- matrix(c(300,212.132,212.132,300),2,2)
  D[2,4] = D[4,2] = D[4,2]*sqrt(1.5)
  Sigma[1,2] = Sigma[2,1] = Sigma[1,2]*sqrt(1.5)

  D <- D *100*delta


  b = mvnfast::rmvn(N, c(0,0,0,0), D)
  ID.ind = c(0,cumsum(n)[-N])
  DataL = mapply(function(i){
    if(frac!=0){
      n1 <- rbinom(1,n[i],0.5)
      if(n1<2){n1=2}
      if(n1>n[i]-2){n1 = n[i]-2}

    }else{
      n1 = n[i]/2
    }

    Treat <- c(rep(-1,n1),rep(1,n[i]-n1))
    X <- cbind(1,Treat)
    Z <- kronecker(diag(2),t(X))
    Mu <- Beta%*%Z
    V <- kronecker(Sigma,diag(n[i]))+t(Z)%*%D%*%Z
    y <- c(mvnfast::rmvn(1,Mu,V))
    Y = matrix(y,ncol=2)
    Endpoint <- c(rep(0,n[i]),rep(1,n[i]))
    data = cbind(i,ID.ind[i]+1:n[i],Treat,Y)
    return(data)
```

```
  },i=1:N,SIMPLIFY = F)
  Data = do.call('rbind',DataL)
  colnames(Data) = c('clusterID','ID','treat','T','S')
  Data = as.data.frame(Data)
  return(Data)
}

# Cluster-by-cluster estimator
CbCEstimator = function(clusterID,Y,X,Z){

  # clusterID: indicator for cluster
  # Y: Matrix of outcomes (rows: observations, columns: variables)
  # X: design matrix X
  # Z: design matrix Z

  ## function to compute overall estimates of Beta
  Weightedestimator = function(K_i,A_i,BetaH_i){
    N = nrow(BetaH_i)
    BetaH_i = split(BetaH_i,1:N)
    ## computing ki'*A*Ki
    Kit = mapply(t,K_i,SIMPLIFY = FALSE)
    KtA = mapply(crossprod,K_i,A_i,SIMPLIFY = FALSE)
    KAK = mapply(tcrossprod,KtA,Kit,SIMPLIFY=FALSE)
    ## computing the inverse
    KAKinv = solve(Reduce('+',KAK))
    ### computing (Ki'*Ai*Ki)^-1*Ki'*Ai
    HHi = mapply('%*%',list(KAKinv),KtA,SIMPLIFY = FALSE)
    ### estimating beta with proportional weights
    BetaTilde = Reduce('+',mapply('%*%',HHi,BetaH_i,SIMPLIFY = FALSE))
    return(list(BetaTilde,HHi))
  }

  Kmatrix = function(X,Z,m){
    if(identical(X,Z)){
      p = ncol(X)
      K = diag(p*m)
    }else{
      Z = kronecker(diag(m),Z)
      X = kronecker(diag(m),X)
      ZtZ = solve(crossprod(Z))
      K = crossprod(ZtZ,crossprod(Z,X))
    }
    return(K)
  }
  prodL = function(XL,YL){
    R = mapply('%*%',XL,YL,SIMPLIFY = FALSE)
    return(R)
  }
  Bmatrix = function(Ki,Inner){
    Ki%*%Inner%*%t(Ki)
  }
  ## Method of moments estimator of D
  MME.D = function(w_i,K_i,HHi,BetaH_i,BetaH,R_i){
```

```
  q = ncol(K_i[[1]])

  N = length(w_i)
  b_i <- BetaH_i- tcrossprod(matrix(1,N,1),BetaH)
  b_i = b_i*matrix(sqrt(w_i),N,q)
  Sb = crossprod(b_i)
  sum.HHi = Reduce('+',HHi)

  H_ii = mapply('%*%',K_i,HHi,SIMPLIFY = FALSE)

  kron.HHi = mapply(function(X,w){w*kronecker(X,X)},HHi,w_i,SIMPLIFY = F)
  kron.K_i = mapply(function(X){kronecker(X,X)},K_i,SIMPLIFY = F)


  Denom.inv = diag((q)^2) - Reduce('+',mapply(function(X,w){w*kronecker(diag(ncol(X)),X)},H_ii,w_i,SIM
    Reduce('+',mapply(function(X,w){w*kronecker(X,diag(ncol(X)))},H_ii,w_i,SIMPLIFY = F)) +
    Reduce('+',kron.K_i)%*%Reduce('+',kron.HHi)

  R_i.w = mapply('*',R_i,w_i,SIMPLIFY = F)
  Inner = Reduce('+',mapply(Bmatrix,HHi,R_i.w,SIMPLIFY = F))
  part1 = prodL(H_ii,R_i.w)
  part2 = prodL(R_i.w,H_ii)
  part3 = mapply(Bmatrix,K_i,list(Inner),SIMPLIFY = FALSE)
  c = Reduce('+',R_i.w) - Reduce('+',part1) - Reduce('+',part2) + Reduce('+',part3)
  vec.c = vec(c)
  Denom = solve(Denom.inv)
  vec.D = Denom%*%(vec(Sb)- vec.c)
  q = ncol(Sb)
  D = invvec(vec.D,nrow=q,ncol=q)
  # browser()
  return(D)
}
## function to estimate the variance of the overall estimator of Beta
VarBetaH.fun = function(Ki,Ai,VarBetai){
  KA = mapply(crossprod,Ki,Ai,SIMPLIFY = FALSE)
  WW = solve(Reduce('+',mapply('%*%',KA,Ki,SIMPLIFY = FALSE)))
  II = Reduce('+',mapply(tcrossprod,mapply('%*%',KA,VarBetai,SIMPLIFY=FALSE),KA,SIMPLIFY = FALSE))
  VarBetaH = WW%*%tcrossprod(II,WW)
  return(VarBetaH)
}

p = ncol(X)
q = ncol(Z)
m = ncol(Y)
Y_i = split(Y,clusterID)
n = c(table(clusterID))
N = length(n)
if(m>1){
  Y_i = mapply(function(x){matrix(x,length(x)/m,m)},Y_i,SIMPLIFY = F)
}
X_i = split(X,clusterID)
if(p>1){
  X_i = mapply(function(x){matrix(x,length(x)/p,p)},X_i,SIMPLIFY = F)
```

```
  }
  Z_i = split(Z,clusterID)
  if(q>1){
    Z_i = mapply(function(x){matrix(x,length(x)/q,q)},Z_i,SIMPLIFY = F)
  }

  Est_i = mapply(function(x){
    Y = Y_i[[x]]
    n = nrow(Y)
    Z = Z_i[[x]]
    BetaH <- solve(crossprod(Z),crossprod(Z,Y))
    e <- Y-Z%*%BetaH
    SigmaH <- crossprod(e)/(n-2)

    Output <- c(c(BetaH),vech(SigmaH))
    return(Output)
  },x=1:N)
  Est_i = t(Est_i)
  BetaH_i = Est_i[,1:(q*m)]
  SigmaH_i = Est_i[,-c(1:(q*m))]
  w_i = n/sum(n)
  w_i.2 = (n-2)/sum(n-2)
  A_i = mapply(diag,w_i,list(q*m),SIMPLIFY = FALSE)
  K_i = mapply(Kmatrix,X_i,Z_i,list(m),SIMPLIFY = FALSE)
  Overall = Weightedestimator(K_i,A_i,BetaH_i)
  HHi = Overall[[2]]
  BetaH = Overall[[1]]
  SigmaH = invvech(apply(SigmaH_i,2,weighted.mean,w=w_i.2))
  # browser()
  R_i = mapply(function(x){
    kronecker(SigmaH,solve(crossprod(Z_i[[x]])))
  },x=1:N,SIMPLIFY = F)
  w_i.temp = rep(1/N,N)
  DH = MME.D(w_i,K_i,HHi,BetaH_i,BetaH,R_i)
  if(min(eigen(DH,only.values = T)$values) < 0 ){
    DH = pdDajustment(DH)
  }
  # browser()
  BetaH1 <- BetaH
  Var.BetaH_i = mapply('+',R_i, MoreArgs = list(DH=DH),SIMPLIFY = F)
  VarBetaH_i.inv = mapply(solve,Var.BetaH_i,SIMPLIFY = FALSE)
  A_i.denom = solve(Reduce('+',VarBetaH_i.inv))
  A_i = mapply('%*%',list(A_i.denom),VarBetaH_i.inv,SIMPLIFY = FALSE)
  Overall = Weightedestimator(K_i,A_i,BetaH_i)
  BetaH = Overall[[1]]
  VarBetaH = VarBetaH.fun(K_i,A_i,Var.BetaH_i)
  Fit = list(BetaH=BetaH,SigmaH=SigmaH,DH=DH,VarBetaH = VarBetaH, BetaH1 = BetaH1)
  # return a list with: (1) Estimates for fixed effects, (2) Estimates Sigma, (3) Estimates D,
  # and (4) variance of estimates for fixed effects
  return(Fit)
}

#### example
```

```
Data = Sim.data(50,100,0.25,1,123)
Y = cbind(Data$T,Data$S)
X = cbind(1,Data$treat)
Z = X
clusterID = Data$clusterID

Est = CbCEstimator(clusterID,Y,X,Z)
Est$BetaH ## estimation of fixed effects

##           [,1]
## [1,] 449.1153
## [2,] 301.1674
## [3,] 499.0259
## [4,] 500.9150

Est$SigmaH # variance of errors

##           [,1]     [,2]
## [1,] 301.8559 261.2946
## [2,] 261.2946 299.6321

Est$DH # variance of random effects

##            [,1]     [,2]     [,3]      [,4]
## [1,] 78.538781 12.36627 19.82643  5.322564
## [2,] 12.366270 76.00486 13.31561 65.633542
## [3,] 19.826432 13.31561 72.24998 13.127020
## [4,]  5.322564 65.63354 13.12702 89.067364

Est$VarBetaH # variance of beta estimators

##            [,1]      [,2]      [,3]      [,4]
## [1,] 1.6400981 0.2491457 0.4564508 0.1080061
## [2,] 0.2491457 1.5895736 0.2678918 1.3727982
## [3,] 0.4564508 0.2678918 1.5138055 0.2643294
## [4,] 0.1080061 1.3727982 0.2643294 1.8503611

Est$BetaH1

##           [,1]
## [1,] 448.4762
## [2,] 300.7024
## [3,] 499.0129
## [4,] 500.7705
```

The R session information (including the OS info, R version and all packages used):

```
sessionInfo()

## R version 4.6.0 (2026-04-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
```

```
## locale:
## [1] LC_COLLATE=Dutch_Belgium.utf8  LC_CTYPE=Dutch_Belgium.utf8
## [3] LC_MONETARY=Dutch_Belgium.utf8 LC_NUMERIC=C
## [5] LC_TIME=Dutch_Belgium.utf8
##
## time zone: Europe/Brussels
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] knitr_1.51  dplyr_1.2.1 ks_1.15.2
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4     tidyr_1.3.2        KernSmooth_2.23-26 shape_1.4.6.1
##  [5] lattice_0.22-9     pracma_2.4.6       lme4_2.0-1         magrittr_2.0.5
##  [9] mitml_0.4-5        evaluate_1.0.5     grid_4.6.0         iterators_1.0.14
## [13] mice_3.19.0        mvtnorm_1.4-2      foreach_1.5.2      jomo_2.7-6
## [17] glmnet_5.0         Matrix_1.7-5       nnet_7.3-20        backports_1.5.1
## [21] tinytex_0.60       survival_3.8-6     mclust_6.1.3       purrr_1.2.2
## [25] codetools_0.2-20   reformulas_0.4.4   Rdpack_2.6.6       cli_3.6.6
## [29] rlang_1.3.0        rbibutils_2.4.1    splines_4.6.0      withr_3.0.3
## [33] otel_0.2.0         pan_2.0            tools_4.6.0        nloptr_2.2.1
## [37] minqa_1.2.8        boot_1.3-32        broom_1.0.13       vctrs_0.7.3
## [41] R6_2.6.1           rpart_4.1.27       lifecycle_1.0.5    MASS_7.3-65
## [45] pkgconfig_2.0.3    pillar_1.11.1      glue_1.8.1         Rcpp_1.1.2
## [49] highr_0.12         xfun_0.60          tibble_3.3.1       tidyselect_1.2.1
## [53] rstudioapi_0.19.0  nlme_3.1-169       compiler_4.6.0     mvnfast_0.2.8
```

```
Sys.time()
```

```
## [1] "2026-08-10 11:11:53 CEST"
```